

풀 버전 라이브 스트리밍 영상을
롱폼 영상으로 요약해주는 서비스



팀명: 아무거나

201724429 김영빈 (kimyb0330@pusan.ac.kr)

202155567 손재우 (jws020319@pusan.ac.kr)

202155573 엄도엽 (yeom5122@pusan.ac.kr)

지도교수: 전상률

제출일: 2026년 5월 15일

부산대학교

목차

1	과제 목표	1
1.1	배경	1
1.2	목표	1
2	대상 문제 및 요구사항	2
2.1	대상 문제	2
2.2	요구사항	2
2.2.1	기능적 요구사항	2
2.2.2	비기능적 요구사항	4
3	현실적 제약 사항 및 대책	5
4	시스템 설계	8
4.1	시스템 아키텍처	8
4.2	프로세스 플로우	10
4.2.1	모델 업데이트 플로우	10
4.2.2	서비스 플로우	12
4.3	데이터베이스	14
4.4	UI 와이어프레임	17
5	한계 및 발전 방향	21
5.1	시각적 분석	21
5.2	편집 가능 포맷	21
6	추진 체계 및 일정	22
6.1	추진 체계	22
6.2	일정	23
7	구성원 구실 분담	24

제1장 과제 목표

1.1 배경

라이브 스트리밍(Live streaming)은 인터넷을 통해 영상 콘텐츠를 실시간으로 전송하는 기술로, 누구나 손쉽게 생방송을 진행할 수 있게 해주었다. 초고속 통신망의 확충과 전자 산업의 발전은 라이브 스트리밍의 진입 장벽을 크게 낮추었다. 또한 기술의 안착과 성숙에 따라 기존 미디어를 크게 위협하는 신세대 매체로 위상이 크게 증대되었다.

이러한 라이브 스트리밍 시장의 급성장으로 인해 방대한 영상 소스가 생성되게 되었고, 이를 가공하는 편집 인력의 비용과 시간 부담이 크게 증가하였다. 실시간으로 진행되는 라이브 스트리밍의 특성상 대부분의 시간은 시청자에게 흥미를 주지 못하며, 이를 가공하여 주요 내용을 모은 편집본이 시청자들에게 일반적으로 선호된다. 따라서 이러한 편집본을 제작하는 영상 편집자의 수요가 크게 늘어나게 된 것이다.

최근 AI의 발전에 따라 영상 편집자의 구실을 대신하는 도구들이 나와 있으나 여기에는 중대한 한계가 있다. 현재의 AI 편집 도구는 2분 이내의 ‘숏폼(Short-form)’ 제작에 집중되어 있으며, 영상 전체의 흐름을 파악하여 10~30분 분량의 ‘롱폼(Long-form)’으로 요약하는 도구는 공백 상태이다. 숏폼 영상만으로는 충분한 내용 전달에 한계가 있기에 롱폼 영상 편집 도구에 대한 수요는 아주 높을 것으로 예상된다.

1.2 목표

과제 수행을 통해 달성하고자 하는 목표는 다음과 같다. 다수의 AI 모델을 활용하여 라이브 풀버전 영상(약 2~16시간)을 10~30분 내외의 요약본으로 자동 편집하는 웹 플랫폼을 개발한다. 요약본의 핵심 내용 판별에는 주로 발언이나 목소리 등 음성 데이터를 사용하며, 채팅과 댓글 등 텍스트 데이터를 추가로 사용한다.

제2장 대상 문제 및 요구사항

2.1 대상 문제

- **편집 효율성:**
장시간 영상을 편집자가 일일이 시청하며 하이라이트를 찾는 비효율적인 업무 과정을 제거한다.
- **문맥 보존:**
단순 컷 편집이 아닌, AI가 영상의 문맥을 이해하여 내용이 이어지는 자연스러운 요약본을 제공한다.

2.2 요구사항

2.2.1 기능적 요구사항

- **URL 입력:**
로그인한 서비스 사용자는 요약하고자 하는 영상의 YouTube URL을 입력하여 대상을 지정한다.
- **영상 파일 획득:**
시스템은 URL이 지시하는 영상 파일을 획득한다.
- **데이터 수집:**
시스템은 YouTube Data API를 통해 채팅 로그, 댓글 타임라인 및 영상 메타데이터 등을 수집한다.
- **멀티모달 분석**
 - **사운드:**
시스템은 OpenAI Whisper 기반 STT(Speech-to-Text) 및 사운드 데시벨 분석을 수행한다.

– 텍스트:

시스템은 시청자 채팅 빈도 밀도 및 댓글 타임스탬프 분석을 수행한다.

• 하이라이트 선별:

시스템은 멀티모달 분석 결과에 가중치를 적용하여 하이라이트로 추정되는 구간의 타임스탬프를 선별한다.

• 자동 인코딩:

시스템은 OpenCV 또는 MoviePy를 활용하여 컷 결합 및 최종 영상 렌더링을 수행한다.

• 자막 추가:

사용자는 편집 결과에 스크립트 내용을 자막으로 추가할지 선택할 수 있다. 만약 옵션이 선택되었다면 시스템은 결과 영상에 자막을 함께 출력한다.

• 결과물 다운로드:

로그인한 사용자는 시스템이 처리한 최종 결과물을 다운로드 받는다.

• YouTube에 업로드:

로그인한 사용자는 시스템이 처리한 결과물을 연동된 YouTube 계정을 통해 업로드 할 수 있다.

• 진행도 확인:

로그인한 사용자는 요청한 작업이 어느 단계에 있는지 확인할 수 있다.

• 회원가입:

사용자는 회원가입을 통해 시스템에 계정을 등록할 수 있다.

• 로그인:

사용자는 로그인을 통해 사용할 계정을 선택할 수 있다.

• 로그아웃:

로그인한 사용자는 로그아웃을 통해 계정 선택을 취소할 수 있다.

• 히스토리 확인:

로그인한 사용자는 지금까지 시스템을 통해 처리한 결과물 목록을 확인할 수 있다.

• 잔여 용량 확인:

로그인한 사용자는 전체 시스템 용량 중 일부를 할당받으며 잔여 용량을 확인할 수 있다.

• 기존 결과물 삭제:

로그인한 사용자는 용량 확보를 위해 기존 결과물을 삭제할 수 있다.

2.2.2 비기능적 요구사항

- **처리 속도:**

시스템은 사용자로부터 요청을 수신 후 결과물을 1시간 이내에 전달할 수 있어야 한다.

- **확장성:**

클라우드 및 Docker 기반 배포로 사용자 증가에 따른 자원 유연성을 확보한다.

- **반응형 웹:**

데스크톱 및 모바일 환경 모두 사용이 가능하도록 반응형 웹페이지를 제공한다.

- **암호화된 통신:**

연결에 HTTPS를 적용하여 사용자-시스템 간 통신이 감청되지 않도록 조치한다.

- **암호화된 사용자 비밀번호:**

사용자 계정 비밀번호에 Salt 및 Hash를 적용하여 정보 유출 시 위험을 제거한다.

- **SQL Injection 방어:**

사용자 입력에 전처리를 적용하여 SQL Injection 공격이 실현되지 않도록 방어한다.

제3장 현실적 제약 사항 및 대책

1. • 제약 사항:

장시간의 영상 데이터를 분석 모델에 그대로 넣으면 자원 부족 현상이 발생하고 시간이 매우 오래 걸릴 수 있다.

• 대책:

무거운 영상 데이터 대신 시청자 반응과 오디오/텍스트를 먼저 분석하여 1~2시간 분량으로 범위를 좁힌다. 시간 단축을 위해 텍스트 기반 편집을 진행한 뒤, 타임스탬프를 포인터로 활용하여 최종 단계에서만 원본 고화질 영상에 적용한다. 작업을 30분~1시간 단위의 청크로 쪼개어 병렬 처리한다.

2. • 제약 사항:

영상 자체를 분석하여 하이라이트를 추출하는 AI 모델을 처음부터 끝까지 구축하는 것은 데이터 확보, 모델 구성, 컴퓨팅 자원 확보, 학습 및 배포에 이르기까지 너무 난이도가 높고 실현 가능성이 떨어진다.

• 대책:

영상에서 활용 가능한 스크립트, 시청자 반응 등의 단서만 추출한 뒤 LLM의 지성과 판단력을 활용하여 충분한 품질을 확보한 하이라이트 부분을 추천하는 시스템을 구성한다.

3. • 제약 사항:

하이라이트 추적을 위해 채팅 데이터를 사용할 시 영상 송출과 채팅 반응 사이에 인지적 지연, 기술적 지연이 발생하게 된다. 이로 인해 정확한 시점을 제시하지 못할 수 있기 때문에 이에 대한 적절한 보정이 필요하다.

• 대책:

슬라이딩 윈도우(Sliding Window) 기법을 활용하여 특정 시간 범위의 채팅 개수를 누적 합산하여 반응 지연을 보정한다.

4. • 제약 사항:

시청자가 많을수록 채팅이 활발해지고, 적을수록 뜸해진다. 같은 스트리밍 내에

서도 시간에 따라 시청자의 수는 계속 변동된다. 채팅 밀도를 근거로 사용하기 위해서는 이에 대한 대책이 필요하다.

- **대책:**

시청자 수 대비 채팅 밀도를 정규화하여 보정 후 사용한다.

5. ● **제약 사항:**

단순히 하이라이트 부분만 추출하여 이어붙인다면 영상이 문맥 없이 흘러가 몰입도를 떨어트릴 수 있다. 요약 결과가 문맥을 보존한 채 컷을 나눌수 있도록 설계해야 한다.

- **대책:**

하이라이트 부분을 중심으로 문맥이 자연스럽게 이어질 수 있도록 필요 시 주변 부분을 포함할 수 있도록 재처리하여 범위를 확장한다.

6. ● **제약 사항:**

장시간의 음성 데이터를 OpenAI Whisper를 통해 분석하면 서비스를 지속하기 어려운 수준의 요금이 부과될 수 있다. 또한 분석 시간 지연에 따라 전체 서비스에 병목이 발생할 수 있다.

- **대책:**

OpenAI Whisper는 출력의 길이가 아닌 입력의 길이에 비례한 요금을 부과한다. 따라서 입력 길이를 줄이기 위해 음성을 2~3배속으로 처리하여 입력한다. OpenAI Whisper는 이러한 입력에 대한 충분한 처리 능력을 가지고 있으며, 처리 시간 단축도 추가로 기대해볼 수 있다.

7. ● **제약 사항:**

많은 영상 데이터를 저장하기 위해서는 큰 용량이 필요하고, 이러한 용량을 클라우드 서비스를 통해 배포하기 위해 많은 비용이 발생할 수 있다.

- **대책:**

Oracle Cloud의 무료 플랜은 200GB의 충분한 용량과 비교적 강력한 컴퓨팅 파워를 제공한다. 조원 1인당 200GB씩, 총 600GB의 용량을 무료로 확보 가능하며 Microsoft Azure의 무료 플랜을 통해 비교적 약한 컴퓨팅 파워의 로드 밸런싱 서버를 구축하여 모든 용량을 활용 가능하도록 구성한다.

8. ● **제약 사항:**

YouTube의 구조가 변경되면 서비스가 동작하지 않을 수 있다. 실제로 YouTube는

광고 차단 등 편법 사용을 막기 위해 조치를 단행하는 일이 잦으며, 영상 다운로드 방식을 변경해야 하는 경우가 발생할 수 있다.

- **대책:**

YouTube 영상 파일을 다운로드 받기 위한 yt-dlp 라이브러리가 Docker 내부에서 구동되도록 구성한다. 그리고 Docker Watchtower를 통해 yt-dlp 컨테이너가 항상 최신 버전으로 유지될 수 있도록 환경을 구축한다.

제4장 시스템 설계

4.1 시스템 아키텍처

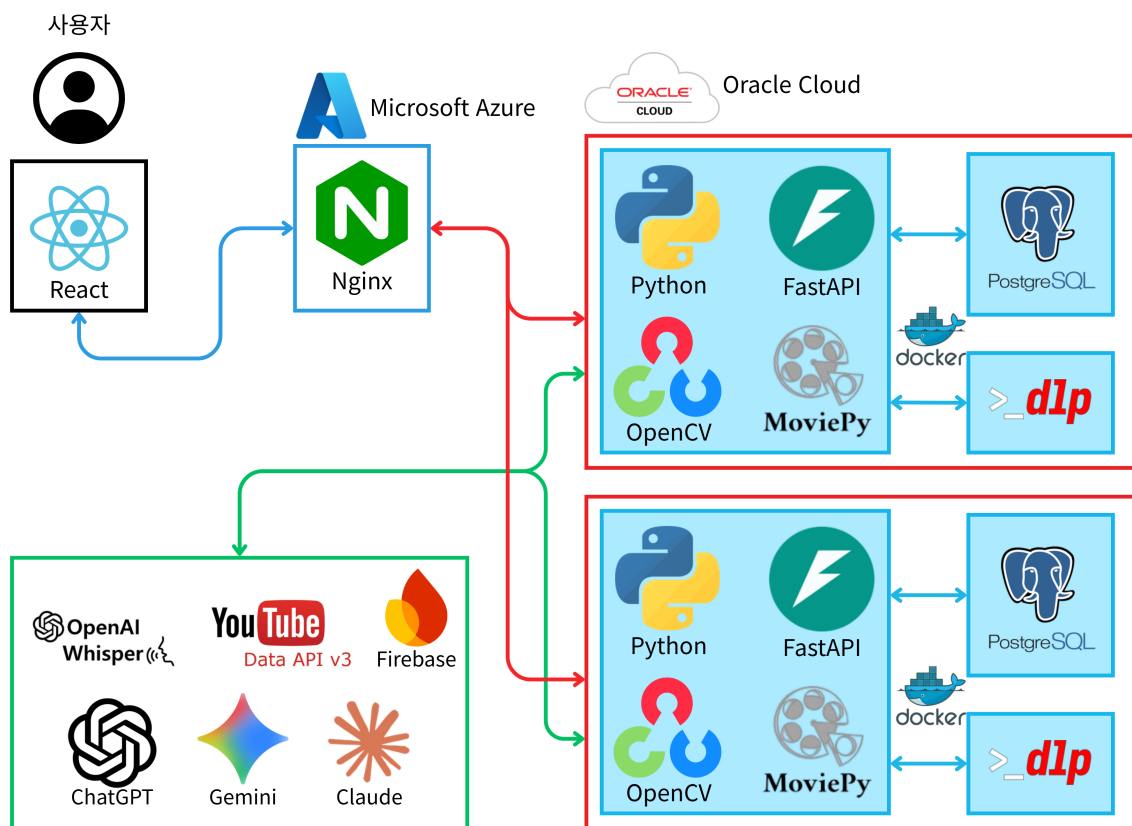


그림 4.1: 아키텍처 다이어그램

- 프론트엔드

- React:

- 사용자에게 서비스 사용을 위한 웹 인터페이스를 제공

- 백엔드

- **Python:**
웹 서버 및 라이브러리 구동을 위한 런타임으로 사용
- **FastAPI:**
웹 서버 구동을 위한 프레임워크
- **OpenCV / MoviePy:**
영상 분할 및 병합을 위한 라이브러리
- **yt-dlp:**
유튜브 영상 파일 다운로드를 위한 라이브러리
- **Postgresql / Firebase:**
각종 정보 저장을 위한 데이터베이스

- **외부 API**

- **OpenAI Whisper:**
영상의 스크립트 추출을 위한 STT 모델
- **YouTube Data API:**
영상의 메타데이터 추출을 위한 API
- **ChatGPT / Gemini / Claude:**
스크립트 외 여러 단서로부터 하이라이트를 판별하기 위해 사용

- **인프라**

- **Docker:**
실행 및 배포 환경 구축을 위한 컨테이너
- **Nginx:**
로드 밸런서 및 리버스 프록시로 사용
- **Microsoft Azure:**
로드 밸런서 구동을 위한 클라우드 1개 인스턴스 사용
- **Oracle Cloud:**
웹 서버 구동을 위한 클라우드 2개 인스턴스 사용

4.2 프로세스 플로우

본 프로젝트의 전체 시스템 흐름은 AI 모델이 하이라이트를 추출하고 하이라이트 근거의 가중치를 조정하는 ‘모델 업데이트 플로우(Model Update Flow)’와, 실제 사용자가 웹 서비스와 상호작용하는 ‘서비스 플로우(Service Flow)’ 두 가지 차원으로 설계했다.

4.2.1 모델 업데이트 플로우

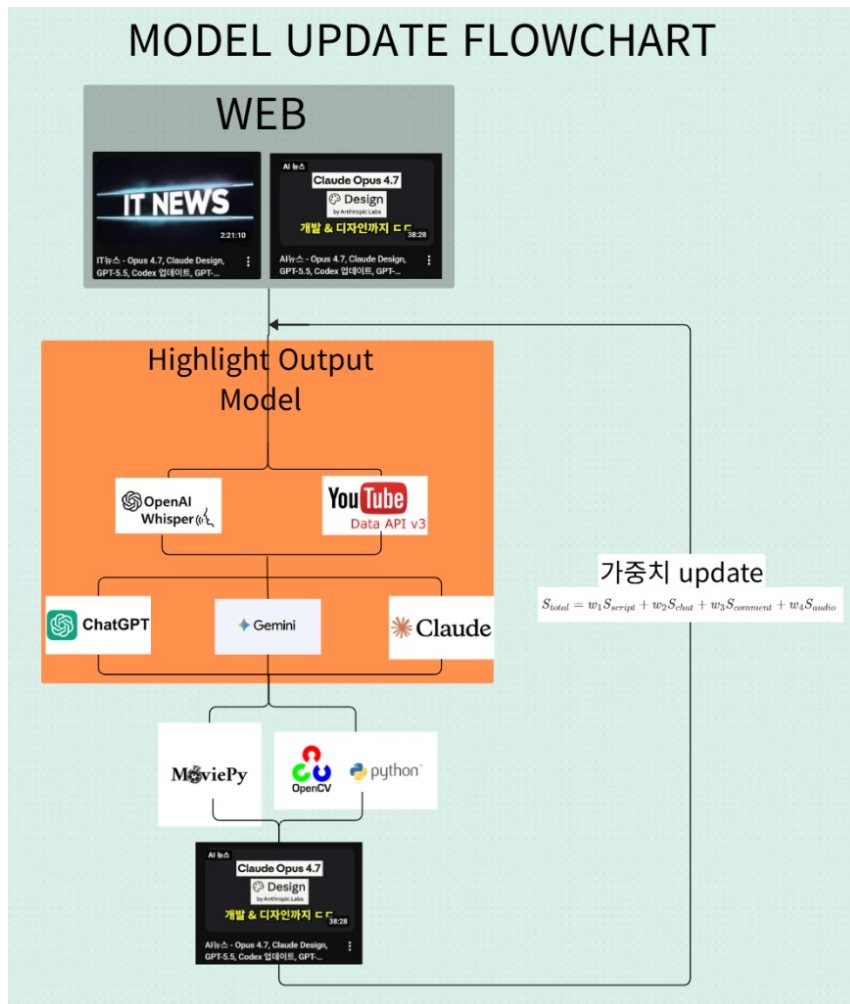


그림 4.2: 모델 업데이트 플로우차트

- 데이터 삽입:

모델의 출력물과 실제 편집본의 비교를 위해 유튜브의 풀 버전 영상과 롱폼으로 편집된 버전의 영상을 함께 넣는다.

- 멀티모달 데이터 수집:

YouTube Data API v3를 통해 시청자 반응(채팅, 댓글)을 수집하고, OpenAI Whisper를 활용해 영상의 오디오를 텍스트(STT)로 변환한다.

- **복합 LLM 평가:**

ChatGPT, Gemini, Claude 등 다양한 LLM을 병렬적으로 활용하여 텍스트의 문맥적 중요도를 평가하고 하이라이트 타임스탬프를 도출한다.

- **하이라이트 영상 추출:**

산출된 최종 타임스탬프를 바탕으로 OpenCV의 장면 전환 분석과 MoviePy를 결합하여 자연스럽게 영상을 컷 및 병합한다.

- **가중치 업데이트 (Weight Update):**

추출의 정확도를 극대화하기 위해 삽입된 풀버전과 편집본을 비교하여 스크립트 문맥(S_{script}), 실시간 채팅(S_{chat}), 댓글 반응($S_{comment}$), 오디오(S_{audio})의 점수를 업데이트 하는 지속적인 피드백 루프를 거친다.

$$S_{total} = w_1 S_{script} + w_2 S_{chat} + w_3 S_{comment} + w_4 S_{audio}$$

4.2.2 서비스 플로우

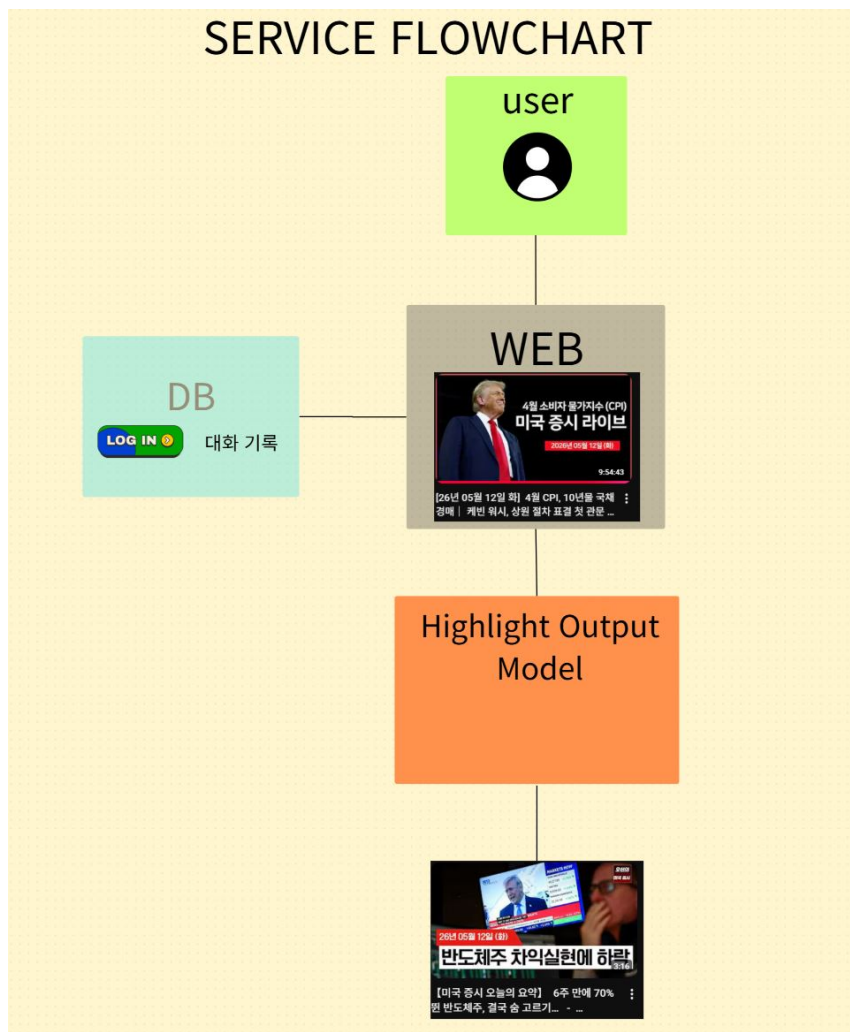


그림 4.3: 서비스 플로우차트

- 사용자 요청 및 DB 연동:

사용자(User)는 웹 페이지에 접속하여 로그인을 수행하며, 이때 데이터베이스(DB)는 사용자의 인증 정보와 AI 모델 사용 기록(대화 및 편집 요청 기록 등)을 통합 관리한다.

- 원본 영상 입력 (WEB):

사용자는 수 시간(2시간 이상의 라이브 방송)에 달하는 라이브 원본 영상의 링크나 파일을 웹 서비스에 입력한다.

- AI 모델 처리 (Highlight Output Model):

웹 서버는 사용자의 요청을 받아 백그라운드의 AI 하이라이트 편집 모델로 전달하여 요약 작업을 수행한다.

- **결과물 반환:**

AI 분석 및 컷 편집이 완료되면, 핵심만 요약된 룻폼 영상(10분~30분 분량)이 웹 대시 보드를 통해 사용자에게 최종 제공된다.

4.3 데이터베이스

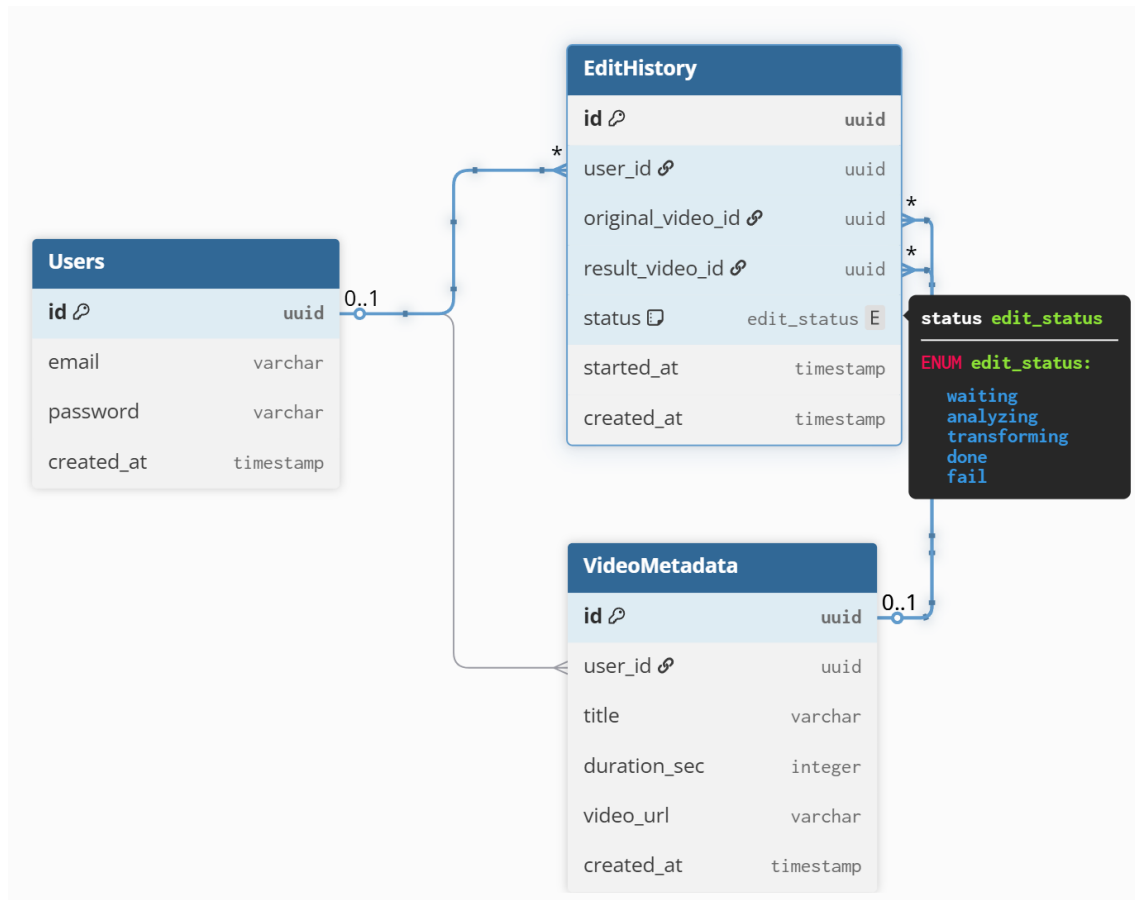


그림 4.4: 스키마 다이어그램

- **Users** (사용자 정보 테이블):

회원가입한 사용자의 계정 정보를 관리하는 테이블이다.

- 식별자(ID) 설계:

UUID를 기본키로 사용하여, 데이터 예측을 통한 개인정보 노출 및 분산 환경에서의 데이터 충돌을 차단했다.

컬럼명	데이터 타입	설명
id	UUID (PK)	사용자 고유 식별자
email	VARCHAR	사용자 이메일 (로그인 ID, UNIQUE)
password	VARCHAR	암호화된 비밀번호
created_at	TIMESTAMP	계정 생성 일시

- **VideoMetadata** (영상 메타데이터 테이블):

사용자가 작업하려는 원본 영상과, AI 편집이 끝난 후 생성된 요약본 영상의 정보를 저장한다.

- **스토리지 분리:**

무거운 동영상 원본 파일을 DB에 직접 넣으면 시스템이 급격히 느려지므로, 실제 파일은 Oracle 등 클라우드 스토리지에 보관하고 DB에는 해당 영상의 URL 링크만 저장한다.

- **병렬 처리의 기준점:**

duration_sec(영상 총 길이)를 저장하여, 12시간 분량의 거대한 영상을 30분 1시간 단위의 청크로 쪼개어 메모리 부족 없이 병렬 처리하기 위한 기준값을 제공한다.

컬럼명	데이터 타입	설명
id	UUID (PK)	영상 고유 식별자
user_id	UUID (FK)	영상을 소유한 사용자 ID (Users 참조)
title	VARCHAR	영상 제목
duration_sec	INTEGER	영상 총 길이 (초 단위, 병렬 처리 분할 기준)
video_url	VARCHAR	영상이 실제 위치한 클라우드 스토리지 URL
created_at	TIMESTAMP	데이터 등록 일시

- **EditHistory** (편집 기록 테이블):

어떤 사용자가 언제 어떤 원본 영상을 편집 요청했고, 현재 AI 작업 상태가 어떠한지, 결과물은 무엇인지에 대한 기록을 통합 관리한다.

- **영상 요청 조회 최적화:**

원본 영상(original_video_id)과 결과 영상(result_video_id)의 관계를 매핑하여, 특정 사용자의 누적 편집 시간이나 월별 사용 통계를 효율적으로 조회할 수 있다.

- **AI 작업 상태 관리:**

status 컬럼에 ENUM 타입(대기 중, 분석 중, STT 변환 중, 완료, 실패 등)을 도입하여, 프론트엔드 화면에서 사용자에게 실시간 작업 진행 상황을 명확하게 업데이트해 줄 수 있다.

컬럼명	데이터 타입	설명
id	UUID (PK)	편집 작업 고유 식별자
user_id	UUID (FK)	작업을 요청한 사용자 ID (Users 참조)
original_video_id	UUID (FK)	편집 대상이 된 원본 영상 ID
result_video_id	UUID (FK)	편집 완료 후 생성된 요약 영상 ID
status	ENUM	현재 AI 편집 진행 상태
started_at	TIMESTAMP	작업(편집 요청) 시작 일시
completed_at	TIMESTAMP	작업 완료 일시

4.4 UI 와이어프레임

4. 랜딩 페이지

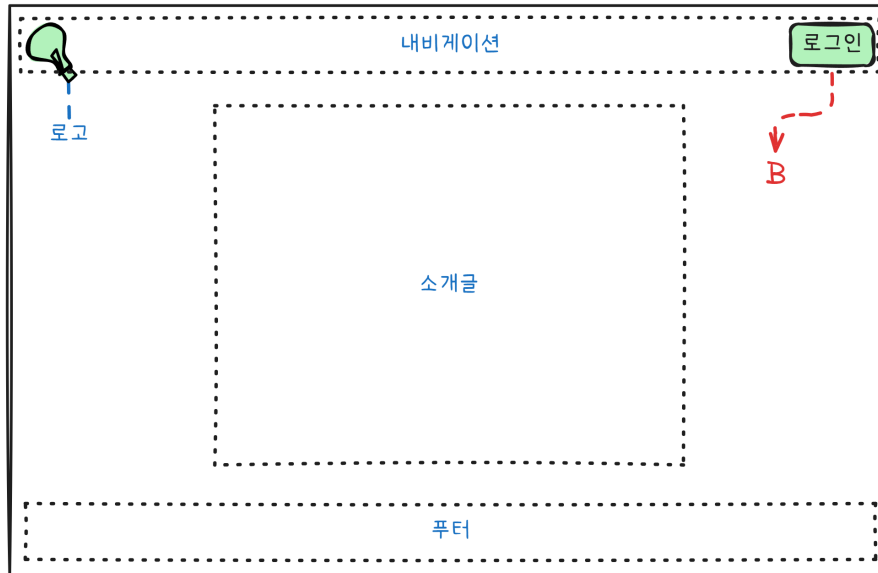


그림 4.5: 와이어프레임 - 랜딩 페이지

B. 로그인 페이지

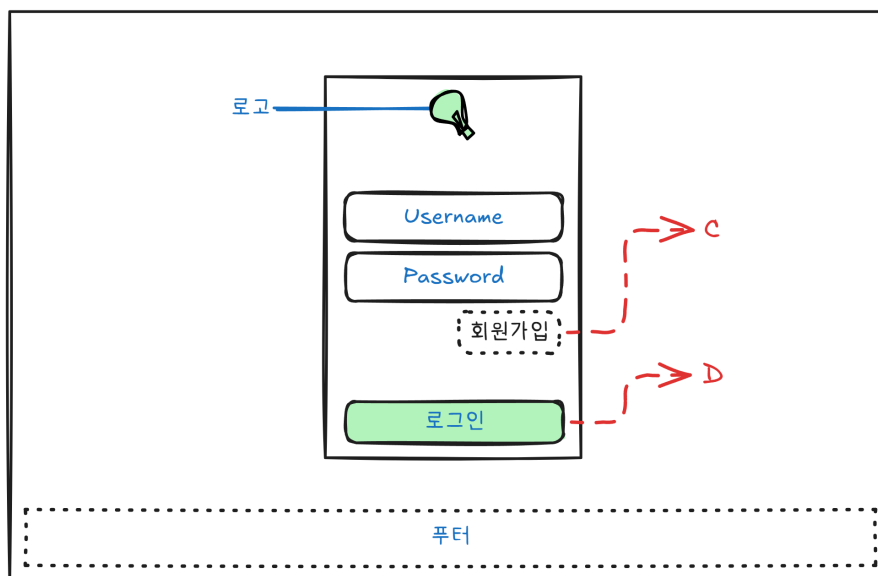


그림 4.6: 와이어프레임 - 로그인 페이지

C. 회원가입 페이지

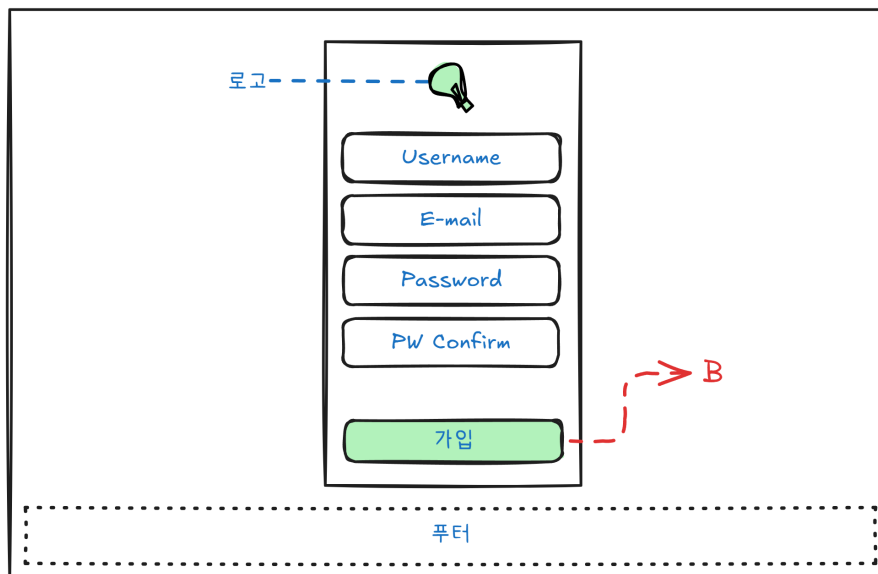


그림 4.7: 와이어프레임 - 회원가입 페이지

D. 영상 요약 요청 페이지

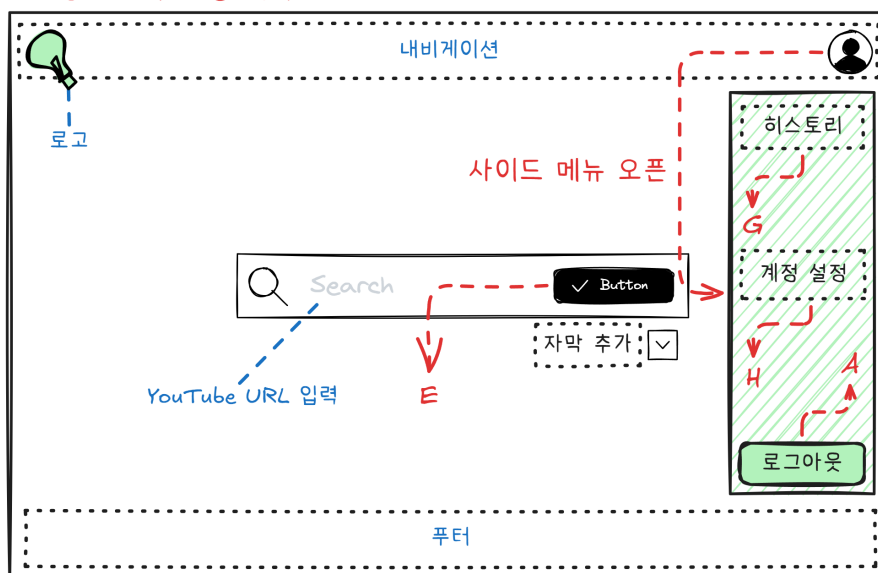


그림 4.8: 와이어프레임 - 영상 요약 요청 페이지

E. 영상 요약 진행도 페이지

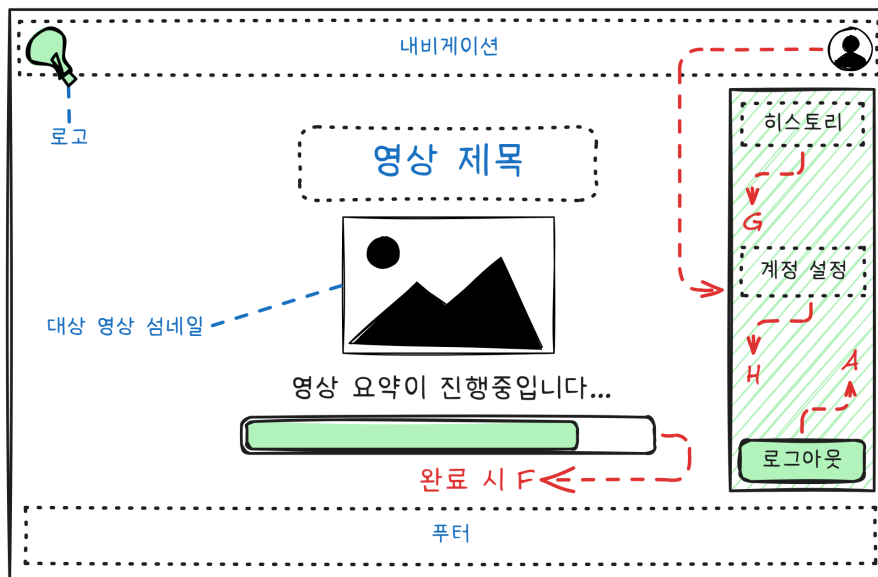


그림 4.9: 와이어프레임 - 영상 요약 진행도 페이지

F. 완료 영상 열람 페이지

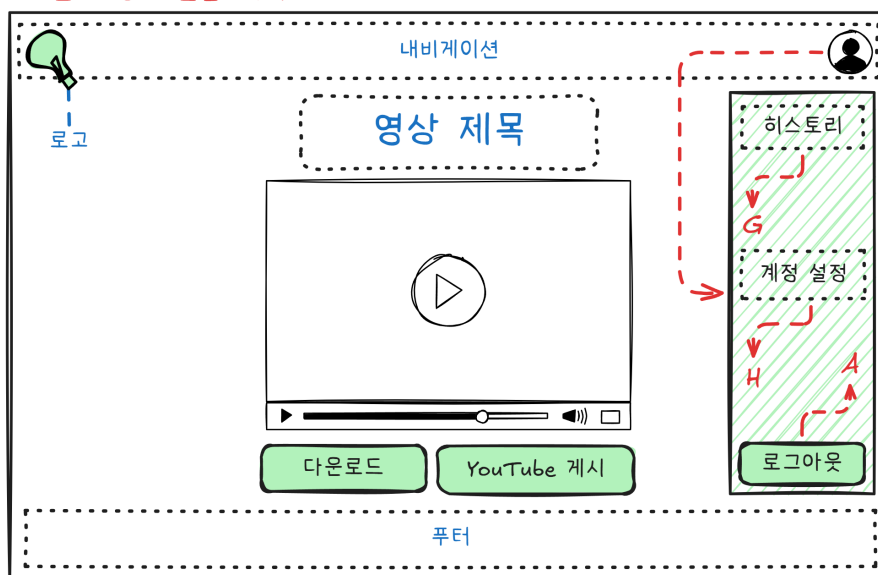


그림 4.10: 와이어프레임 - 완료 영상 열람 페이지

G. 히스토리 열람 페이지

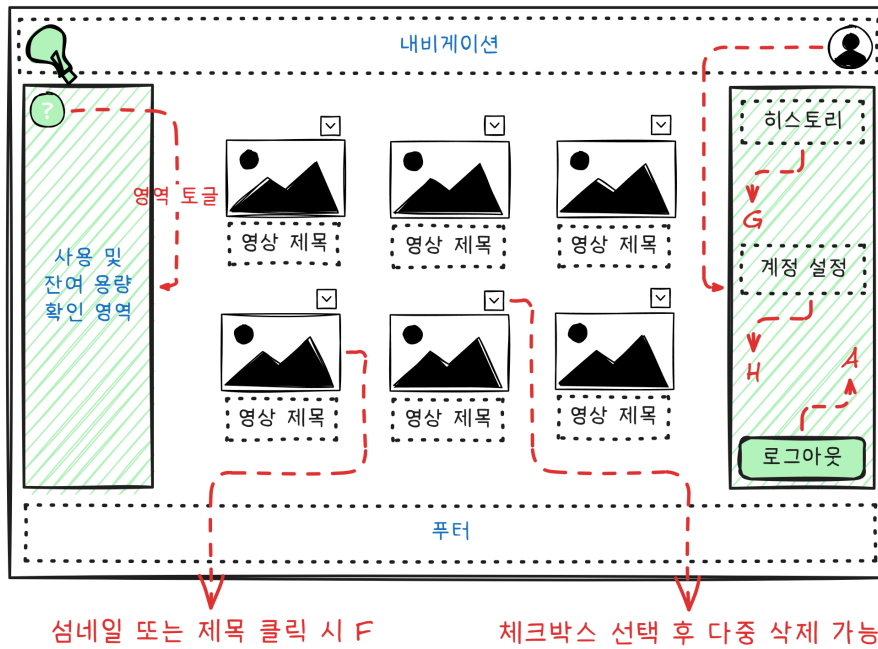


그림 4.11: 와이어프레임 - 히스토리 열람 페이지

H. 계정 설정 페이지

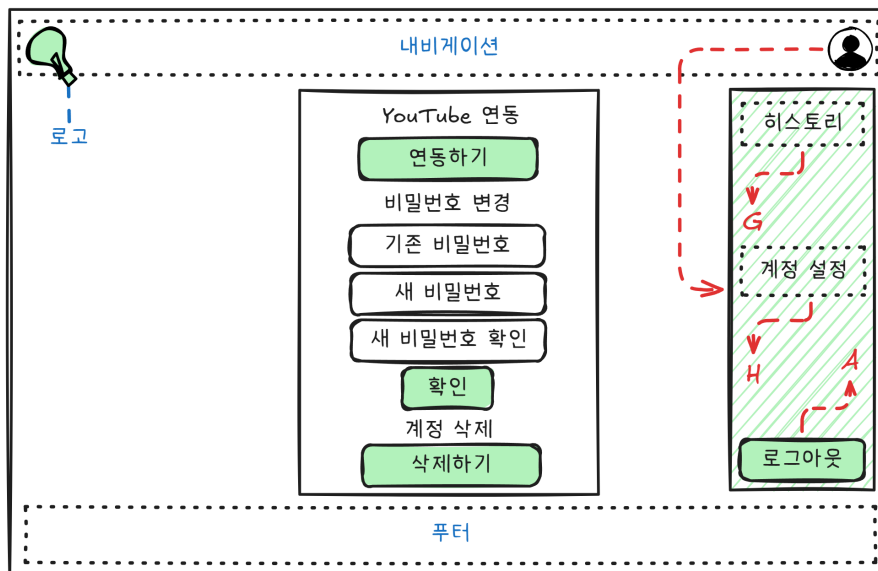


그림 4.12: 와이어프레임 - 계정 설정 페이지

제5장 한계 및 발전 방향

5.1 시각적 분석

현재의 시스템은 구조상 음성 및 텍스트 데이터만을 가지고 하이라이트를 추출하는 기능을 가진다. 그러나 모든 라이브 스트리밍에서 충분한 음성과 텍스트 데이터가 제공되는 것은 아니다. 또한 다른 일반 영상에 대한 요약 기능을 확보하기 위해서는 이 외의 다른 데이터도 분석할 수 있는 능력을 갖추어야 한다.

이와 같이 더 넓은 범위의 영상을 지원하기 위해서 시각 데이터에 대한 분석 능력을 갖추어야 할 필요성에 대해 생각해볼 수 있다. 기존 방식대로 LLM을 통한 판단이라는 기본 구조를 유지하기 위해 이미지로부터 문맥을 파악하여 텍스트로 반환하는 방법을 생각해볼 수 있다. OpenAI의 CLIP 모델 등을 사용하면 이미지를 텍스트 공간으로 임베딩하여 기능을 확장하는 것이 가능할 것으로 생각된다. 추후 이러한 구조를 추가할 수 있다면 본 시스템의 활용 범위가 훨씬 넓어질 수 있을 것으로 기대된다.

5.2 편집 가능 포맷

현재 시스템의 최종 출력물은 편집 및 병합이 완료된 단일 영상 파일이다. 이 영상의 품질은 완벽하지 않고 사용자에게 충분한 만족을 주지 못할 수 있다. 이 때 결과물을 사용자 입맛대로 수정이 가능하다면 작업 시간은 줄어들면서 만족스러운 결과를 얻을 수 있을 것이다.

하지만 현재 구조대로 단일 영상 파일만을 제공한다면 편집을 위한 충분한 환경을 제공하지 못할 것이다. 여기서 하이라이트 추출 후 인코딩이 되기 이전의 편집 가능한 포맷을 제공할 수 있다면 유용함과 사용자 만족도를 극적으로 끌어올릴 수 있을 것이다. 이러한 포맷은 컷 분할 및 자막 편집이 가능한 XML 포맷이 주로 사용된다. 추후에 이를 구현할 적절한 방법을 발견하게 된다면 해당 기능을 지원하는 것을 적극 검토할 가치가 있어 보인다.

제6장 추진 체계 및 일정

6.1 추진 체계

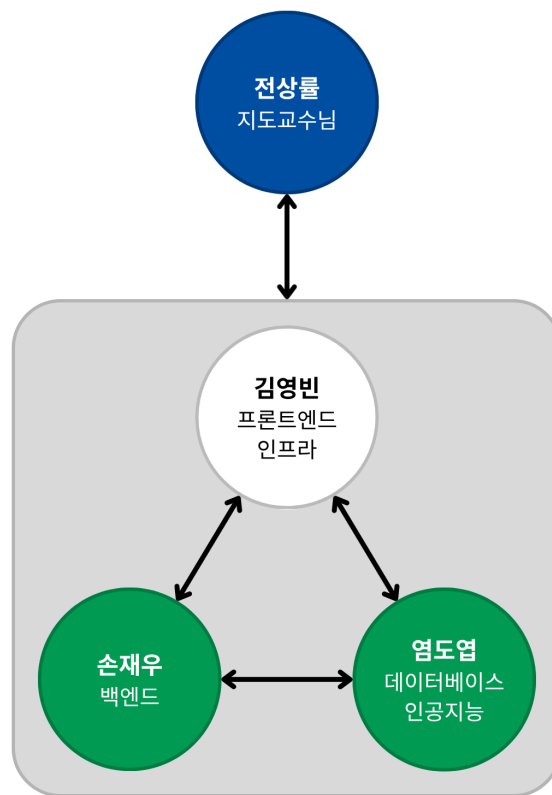


그림 6.1: 추진 체계도

6.2 일정

구분	추진 목표	5월				6월				7월				8월				9월			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
기획	요구사항, API 및 아키텍처 설계																				
	기능 구체화																				
	작성 보고서 작성																				
	AI 다단계 파이프라인 기획																				
	중간 보고서 작성																				
	최종 보고서 작성																				
AI 모델 개발	채팅/댓글 빈도 기반 1차 필터링 모직																				
	Whisper STT 모델 연동 및 대서 추출																				
	LLM(Gemini/GPT) 문맥 요약 포맷포팅																				
	OpenCV/MoviePy 영상 컷 및 병합																				
	모델 디버깅 및 영상 길이 조절 파라미터 수정																				
백엔드 개발	FastAPI 서버 및 AI 파이프라인 연동																				
	Firestore Auth 및 DB 구현																				
	체크 분할 및 비동기 처리																				
프론트 개발	UI/UX 화면 설계																				
	실시간 편집 상태 업데이트 UI																				
	요약 영상 결과 조회 대시보드																				
배포 및 테스트	클라우드 환경(Oracle) 서버 배포																				
	영상 처리 속도 최적화 및 최종 디버깅																				

표 6.1: 일정 Gantt Chart

제7장 구성원 구실 분담

이름	구실
김영빈	프론트엔드 인프라
손재우	백엔드
염도엽	데이터베이스 인공지능

표 7.1: 구성원 구실 분담 요약표

- 김영빈 (조장)
 - UI/UX 화면 설계
 - 실시간 편집 상태 업데이트 UI
 - 요약 영상 결과 조회 대시보드
 - 클라우드 환경 서버 배포
- 손재우
 - OpenCV/MoviePy 영상 컷 및 병합
 - FastAPI 서버 및 AI 파이프라인 연동
 - 청크 분할 및 비동기 처리
 - 영상 처리 속도 최적화 및 최종 디버깅
- 염도엽
 - 채팅/맷글 빈도 기반 1차 필터링 로직
 - Whisper STT 모델 연동 및 대사 추출
 - LLM 문맥 요약 프롬프팅

- 모델 디버깅 및 영상 길이 조절 파라미터 수정
- Firebase Auth 및 DB 구현

- **공통**

- 요구사항, API 및 아키텍처 설계
- 기능 구체화
- AI 다단계 파이프라인 기획
- 착수 보고서 작성
- 중간 보고서 작성
- 최종 보고서 작성